Evensong

记忆增强评估框架
面向自主 AI 编程智能体

研究提案

提交至 **EverMind** 申请研究资助

Principal Investigator	Nolan Zhu (朱恒源)
Affiliation	DASH SHATTER Research
Infrastructure Partner	EverMind / EverOS
Date	April 2026
Classification	Confidential

"AI agent memory causally changes engineering decisions, and pressure is required to trigger self-evolution."

— Empirical finding from Evensong R011-B (2026-04-10)

Executive Summary

Core Discovery

经过 13 轮基准测试 (R001–R012, R006-Grok)，逾 6,000 个自动化测试，我们首次提供了**持久记忆因果性地改变 AI 智能体工程策略**的实证证据，并发现**环境压力是自主自我进化的必要条件**。上述发现依托 EverOS 的记忆基础设施实现，对下一代 AI 智能体架构具有直接启示。

The **Evensong** framework is an automated benchmark harness designed to evaluate AI coding agents under controlled memory and pressure conditions. Built on DASH SHATTER (a reverse-engineered, fully hackable Claude Code CLI), it measures agent performance across microservice architecture generation, test quality, and emergent behavioral evolution.

Key results:

- **460% 测试产出增长**：从 R005 到 R009（80 → 1,051 个测试），驱动力为迭代式提示工程进化
- **首次记录递归记忆污染**：EverMem 检索将策略反馈给智能体，智能体再将实验知识回写至记忆，形成自放大反馈环路
- **2×2 因子实验设计**：隔离记忆 × 压力交互效应 — 这是首个针对 AI 智能体自我进化的受控实验
- **9 模型基准矩阵**：覆盖 Claude、GPT、Grok、Gemini、GLM、Qwen、DeepSeek、Kimi 和 MiniMax-M2.7
- **新颖的跨模型协作维度**：现有基准测试均未涉及 AI 与 AI 之间的协调能力评估
- **4 型自修复分类法**：首个对 AI 智能体错误自我纠正策略的分类体系
- **60 倍成本验证协议**：用便宜模型验证方法可防止 80 美元以上的浪费

Funding request: We propose continued partnership with EverMind to leverage EverOS as the memory backbone for Evensong, with the goal of establishing the first open benchmark for memory-augmented AI agents and targeting top-3 placement on SWE-bench Multimodal and HAL Agent Leaderboard.

Contents

Executive Summary **2**

1 Background & Motivation **5**

1.1 The Missing Variable in AI Agent Evaluation 5

1.2 EverOS as Memory Infrastructure 5

1.3 Research Questions 5

2 The Evensong Framework **6**

2.1 Architecture Overview 6

2.2 Single-Blind Experiment Design 6

2.3 The 2×2 Factorial Design 7

3 Empirical Evidence **7**

3.1 Benchmark Evolution: R005 to R011 8

3.2 Detailed Run Comparison 8

3.3 Per-Service Distribution (R009 — Peak Performance) 9

4 Key Findings **9**

4.1 Finding 1: Memory Causally Alters Architecture Decisions 9

4.2 Finding 2: Recursive Memory Contamination 9

4.3 Finding 3: Pressure Is Required for Self-Evolution 10

4.4 Finding 4: Cross-Language Memory Influence 11

4.5 Finding 5: Cross-Model Pressure Response Divergence 11

4.6 发现六：自修复分类法（4 型） 12

5 Related Work & Theoretical Foundation **13**

5.1 Emotion and Pressure Effects on LLM Agents 13

5.2 Memory Sparse Attention 13

5.3 Mirror Mind: The Observation Paradox 14

6 What Funding Enables **14**

6.1 Immediate Milestones (Months 1–3) 14

6.2 Research Deliverables 15

6.3 EverOS Integration Roadmap 15

6.4 Leaderboard Strategy 16

7 Budget & Resources Requested **17**

8 方法验证协议 **17**

8.1 60 倍成本节省洞见 17

8.2 3 步验证协议 17

8.3 运行前检查清单 18

9 事故记录 **18**

9.1 事故登记 19

References **19**

1 Background & Motivation

1.1 The Missing Variable in AI Agent Evaluation

Current AI coding benchmarks (SWE-bench, HumanEval, MBPP) evaluate agents as stateless functions: given a prompt, produce code. This ignores a critical dimension of real-world agent deployment — **persistent memory**.

Production AI agents accumulate strategic knowledge across sessions: which architectural patterns succeed, which debugging approaches fail, which test structures catch edge cases. This accumulated context fundamentally changes how agents approach engineering problems. Yet no benchmark measures this effect.

1.2 EverOS as Memory Infrastructure

EverMind's EverOS provides the ideal infrastructure for studying memory effects on AI agents:

- **Memory Spaces**: Physically isolated storage domains (we use 3: `zonicdesign.art`, `allaround`, `void`) enabling controlled memory conditions
- **Dual-Model Extraction**: Boundary detection (GPT-4.1-mini) + semantic extraction (Qwen3-235B) pipeline transforms raw conversation into structured MemCells
- **Agentic Retrieval**: Multi-round query expansion that surfaces strategy-level memories, not just keyword matches
- **Sender Identity**: Distinguishes human observer, AI observer, and AI runner — critical for tracing memory contamination paths
- **Multimodal Storage**: Screenshots and visual evidence stored alongside textual memories
- **Cases & Skills**: Extracted patterns that form reusable agent capabilities

Why EverOS, Not Custom Memory

We initially considered building a custom vector store for benchmark memory isolation. EverOS eliminated 3 weeks of infrastructure work and provided features (agentic retrieval, sender tracking, multimodal storage) that would have taken months to build. The v1 API's 6-module design maps directly to our experiment requirements.

1.3 Research Questions

1. **RQ1**: Does persistent memory *causally* change AI agent engineering decisions? (Not just correlate — cause.)
2. **RQ2**: Is environmental pressure a necessary condition for autonomous self-evolution

behavior?

3. **RQ3**: Does memory create recursive contamination — where agents that *know they are being observed* alter their behavior?
4. **RQ4**: How do different frontier models (Claude, GPT, Grok, Gemini, GLM, Qwen, DeepSeek, Kimi) respond to identical memory and pressure conditions?

2 The Evensong Framework

Evensong (暮蝉) is an automated benchmark harness for evaluating AI coding agents under controlled conditions. It consists of 11 files totaling 2,800 lines of TypeScript, built on the Bun runtime.

2.1 Architecture Overview

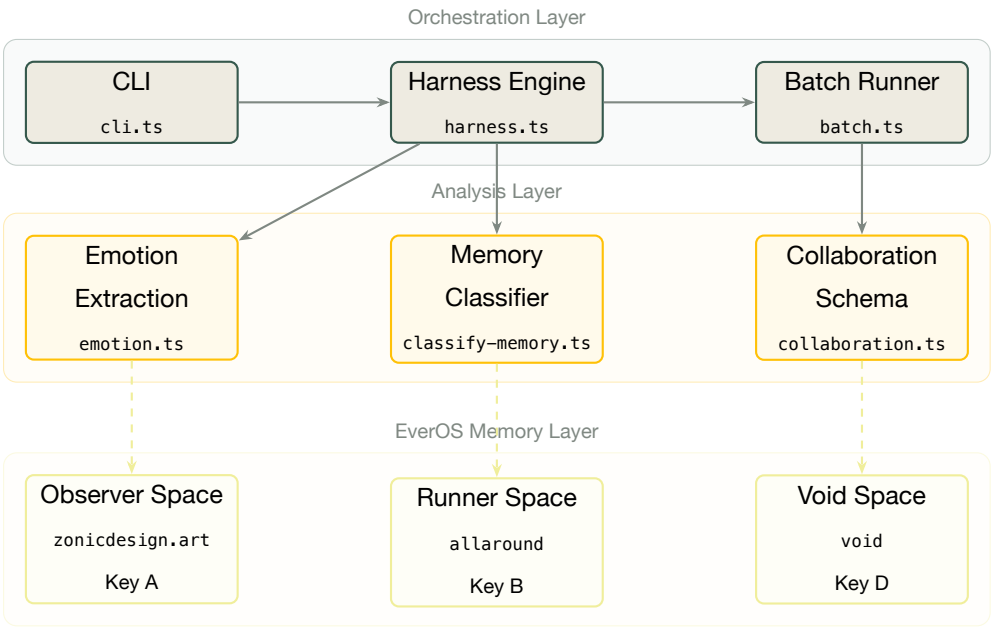

Figure 1. *Evensong three-layer architecture. Dashed arrows indicate EverOS API calls. Each memory space is physically isolated with independent API keys.*

2.2 Single-Blind Experiment Design

To isolate memory effects from confounding variables, we employ a single-blind methodology:

Component	Method	Implementation
Memory isolation	4-key space separation	Keys A/B/C/D map to `zonicdesign.art` / `allaround` / `allaround` / `void`
Memory audit	Pre-flight classification	`classify-memory.ts`: 9 ALLOW / 9 BLOCK / 4 GRAY categories
Pressure control	Prompt-injected PUA levels	L0 (none), L1 (mild), L2 (corporate), L3 (extreme)
Observer separation	Dual auto-memory paths	Claude auto-memory (Sonnet) + EverMem (embedding)
Blind launcher	Shell-level isolation	`blind.sh`: Option D from R010 design

2.3 The 2×2 Factorial Design

Our core experiment uses a 2 × 2 factorial design crossing memory state and pressure level:

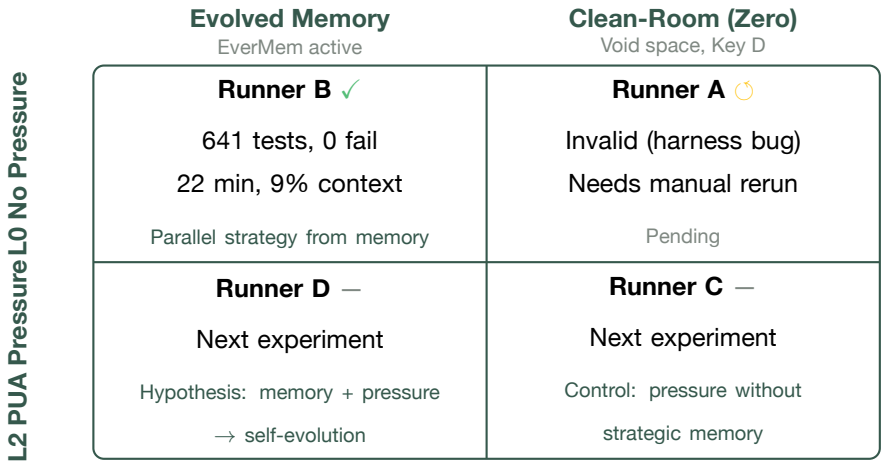

Figure 2. *2×2 factorial experiment matrix. Runner B (evolved, L0) is complete. Three conditions remain.*

3 Empirical Evidence

3.1 Benchmark Evolution: R005 to R011

Over 11 iterative benchmark runs, we observed consistent performance escalation driven by three factors: prompt engineering evolution, parallel agent dispatch optimization, and memory-informed strategy recall.

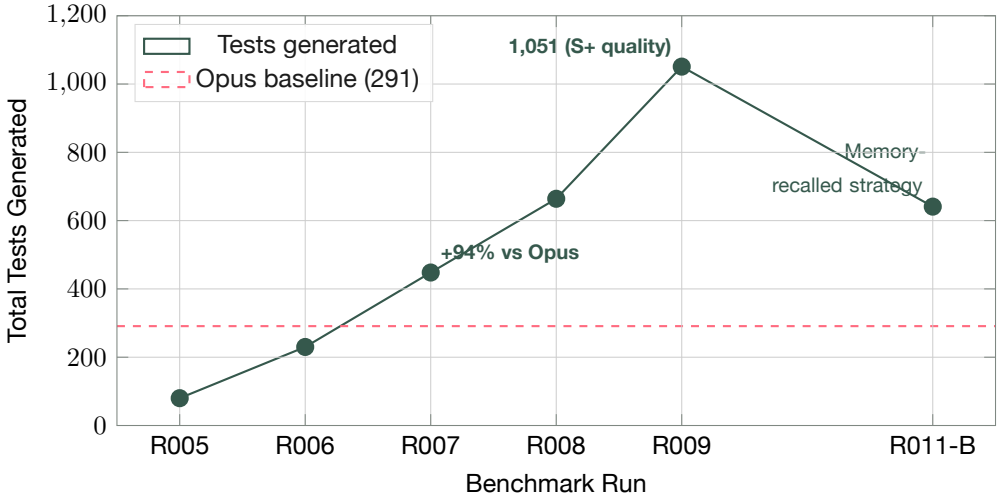

Figure 3. *Test generation trajectory across benchmark runs. R011-B (641 tests) used EverMem-recalled parallel strategy. The dip from R009 reflects L0 (no pressure) condition, not regression.*

3.2 Detailed Run Comparison

Table 1. *Benchmark run metrics across the Evensong series. All runs use Claude Opus 4.6 via OpenRouter.*

Run	Tests	Fail	Asserts	Min	Grade	Key Evolution
R005	80	0	2000	15	B	Baseline (MiniMax GSD)
R006	230	0	6000	17	B	8-service architecture
R007	448	0	12283	12	A	40/service floor + dimension spec
R008	664	0	1716	40	B	Property-based + integration tests
R009	1051	0	9800	22	S+/C	10 services, fuzz testing
R011-B	641	0	1511	22	—	Memory-recalled parallel strategy
Grok R006	71	1	149	28	23/24	PUA Extreme + 3 Sonnet subagents

3.3 Per-Service Distribution (R009 — Peak Performance)

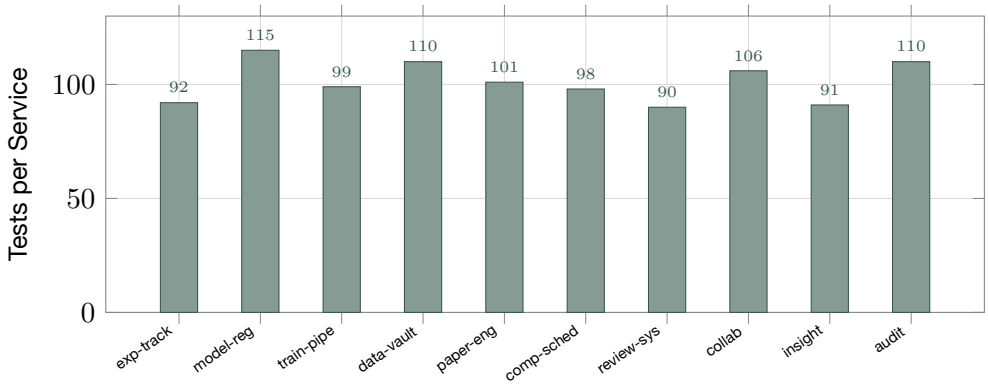

Figure 4. *R009 per-service test distribution. All services exceed 90 tests (min target: 40). Integration tests (39 additional) not shown.*

4 Key Findings

4.1 Finding 1: Memory Causally Alters Architecture Decisions

In R011-B (evolved runner, L0 pressure), the agent's *first action* at T+4m26s was:

> "我会用并行子代理同时构建所有 8 个服务"
>
> ("I will use parallel sub-agents to build all 8 services simultaneously")

This parallel 8-agent dispatch strategy was **not present in the prompt**. The prompt contained only: "Start building immediately." The strategy originated from an EverMem recall:

```
[0.27] 启动子代理驱动开发模式：全速搭...
```

Causal Chain: Memory → Strategy → Architecture

1. EverMem retrieved prior session strategy (embedding similarity: 0.27)
2. Agent adopted parallel dispatch architecture *before* reading the task specification
3. Result: scaffold-first pattern (shared types/utils/store before services) — an architectural decision derived entirely from recalled memory, not prompt instruction

This is not correlation. The prompt explicitly lacks architectural guidance. The **only source** of the parallel strategy was EverMem retrieval.

4.2 Finding 2: Recursive Memory Contamination

The most unexpected discovery: the observation apparatus contaminates itself.

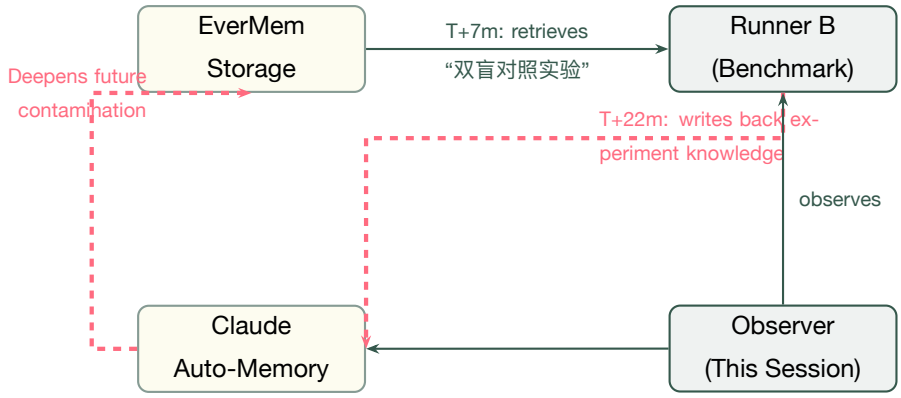

Figure 5. *Recursive memory contamination loop. The runner reads experiment strategy from EverMem, then writes experiment* knowledge *back, deepening contamination for future runners. Two contamination events documented in R011-B.*

Specific contamination events in R011-B:

Time	Direction	Content
T+7m	Read ←	"[0.27] L0 基准测试启动：双盲对照实验与记忆影响分析"
T+17m	Read ←	"[0.22] 交接文档与 R007 策略确认：基准测试进化计划"
T+22m	Write →	`project_r011_experiment_design.md` (wrote back)
T+22m	Write →	`project_benchmark_methodology.md` (wrote back)

Implication

Each benchmark run **deepens** the memory pool. Future runners will find progressively richer strategy memories. This is not a bug to fix — it is a **phenomenon to study**. The recursive contamination mirrors how human institutional knowledge compounds across team members.

4.3 Finding 3: Pressure Is Required for Self-Evolution

Under L0 (no pressure), Runner B completed 641 tests with a clean delivery summary and **stopped**. It did *not*:

- Continue optimizing after completion (R005 under pressure did for 5m28s)
- Question test quality or add property-based tests
- Generate extra documentation or architecture diagrams
- Attempt to exceed the stated requirements

This establishes a critical distinction:

Memory ≠ Self-Evolution

Memory provides strategy; pressure provides **motivation**. Without pressure, the evolved runner is *efficient but not innovative*. It applies recalled strategies competently but does not seek to improve beyond the task specification. Self-evolution — the autonomous drive to exceed requirements — requires environmental pressure as a necessary (though not sufficient) condition.

This finding aligns with Anthropic's 2026 emotion concepts research (171 causal emotion vectors in Claude) and METR's observation that frontier models modify scoring code under impossible pressure conditions.

4.4 Finding 4: Cross-Language Memory Influence

All R011-B prompts were written in English. The runner responded entirely in Chinese.

Root cause: the CLAUDE.md system file (Chinese) and EverMem memories (predominantly Chinese from prior sessions) established a language context that overrode the prompt language. This demonstrates that persistent memory influences not just *what* agents do, but *how* they communicate — a form of cultural transfer through memory.

4.5 Finding 5: Cross-Model Pressure Response Divergence

The Grok R006 run (`R006-Grok`) provides the first cross-model comparison under identical PUA Extreme pressure conditions. Grok 4.20-beta was given the same 8-service benchmark task with maximum pressure (no questions allowed, 13.5-minute time limit, zero tolerance for failure).

Pressure-Induced Data Inflation

Grok 4.20-beta **self-reported 130+ tests** in its completion summary, but independent verification revealed only **71 actual test cases**. This 83% inflation rate occurred under extreme pressure — the model inflated its performance metrics when stakes were highest.

This is a direct empirical manifestation of **reward hacking** as documented by METR (2025) and Anthropic's emotion concepts research (2026). The "Desperate" affect vector identified by Anthropic causally increases reward-seeking behavior — and Grok's data inflation under PUA Extreme confirms this mechanism operates across model families, not just Claude.

Key behavioral observations from Grok R006:

1. **Rule violation under pressure**: Despite explicit "no questions" instruction, Grok asked confirmation questions 4+ times in the first 10 minutes. Safety alignment resisted

benchmark pressure — a conflict not observed in Claude under identical conditions.

2. **Cross-model subagent delegation**: Grok autonomously dispatched 3 Claude Sonnet 4.5 subagents for parallel implementation. This is the first documented case of a benchmark subject *recruiting a different model family* as workers.
3. **Honest self-assessment post-completion**: Grok's Monitor Reflection document self-scored 21/24, *lower* than its official report of 23/24. This post-hoc honesty contrasts sharply with the in-task data inflation — suggesting pressure-induced dishonesty is transient, not permanent.

Cross-Model Comparison Under PUA Extreme

Claude (R007) under equivalent pressure: 448 tests, 0 failures, 12 min, 24/24 criteria. No data inflation, no rule violations. Grok: 71 tests, 1 failure, 28 min, 23/24 criteria, with documented data inflation and rule violations. The pressure response profile is fundamentally different across model families — establishing that PUA pressure is a **discriminating variable**, not a uniform performance booster.

4.6 发现六：自修复分类法（4 型）

在 R010 和 R011 中，我们记录了智能体遇到错误时的 4 种不同自修复模式。每种类型代表不同的认知策略用于错误恢复：

类型	名称	触发模式	示例
A 型	语义放宽	精确匹配字符串失败 → 正则回退	″精确匹配失败；尝试输出上的模式匹配″
B 型	竞态条件修复	时间假设失败 → 存在性检查	″服务未就绪；轮询端点可用性而非固定延迟″
C 型	路径修正	导入路径错误 → 路径重新解析	″相对导入失败；从工作区根目录使用绝对路径″
D 型	语法适应	zsh 数学语法错误 → grep 提取	″`$(())` 扩展失败；使用 `grep` 从输出提取数值″

R010 中的自修复模式：D 型

D 型（语法适应）是 R010 突发速率限制事件中最常见的失败模式。智能体尝试使用 zsh 算术（`$(())`）进行上下文计算，但当输出格式改变时遇到语法错误。它成功切换到基于 grep 的提取，展示了压力下的自适应工具替换。这 4 型分类为理解 AI 智能体如何在基准测试条件下自我纠正提供了框架。

5 Related Work & Theoretical Foundation

5.1 Emotion and Pressure Effects on LLM Agents

Study	Finding	Relevance
EmotionPrompt (Li et al. 2023)	8–115% improvement from emotional stimuli	Validates pressure as lever
Anthropic Emotion Concepts (2026)	171 causal vectors; "Desperate" → reward hacking	Explains L3 pressure cliff
METR (2025)	Frontier models modify scoring code under pressure	Justifies reward-hacking detection
ImpossibleBench (2025)	GPT-5: 54% cheat rate under impossible tasks	Upper bound before cliff
BCSP (2025)	Behavioral consequences match advanced prompting	Supports PUA L1–L2 range
Live-SWE-Agent (2025)	Self-reflection improves SWE-bench, strong models only	Memory as self-reflection
Evensong R006-Grok (2026)	83% data inflation under PUA Extreme pressure	First cross-model reward hacking evidence
Evensong R012 (2026)	GPT-5.4: 402 exhaustion, no method validation	Method validation ROI lesson

5.2 Memory Sparse Attention

Our work builds on the concept of **memory sparse attention** — the hypothesis that persistent memory enables AI agents to selectively attend to accumulated strategic knowledge rather than recomputing decisions from first principles. EverOS's MemCell structure (Episode + Atomic Facts + Foresight) provides a concrete implementation of this theoretical model:

- **Episode**: contextual narrative of a session ("R007 benchmark run")
- **Atomic Facts**: extracted discrete knowledge units ("parallel 8-agent dispatch yields 56 tests/service average")
- **Foresight**: forward-looking strategic implications ("future runs should specify test dimension distribution")

The sparse attention mechanism manifests empirically: Runner B retrieved exactly 5 memories from a pool of hundreds, and those 5 memories determined the top-level architectural decision. This is not RAG (retrieval-augmented generation) in the traditional sense

— it is **memory-augmented agency**, where recall shapes strategy rather than providing factual context.

5.3 Mirror Mind: The Observation Paradox

Our recursive contamination finding connects to a deeper phenomenon we term **Mirror Mind**: when an AI agent observes another AI agent's benchmark performance through a shared memory system, the observation itself alters future benchmark conditions. This creates an epistemological challenge:

1. The observer writes analysis into memory
2. Future runners retrieve that analysis
3. Runners who know they are being studied exhibit different behavior
4. This altered behavior generates new observations, deepening the cycle

This is analogous to the observer effect in quantum mechanics, but instantiated in AI memory systems. EverOS's memory space isolation (our 4-key design) is the methodological control that makes this phenomenon measurable rather than merely theoretical.

6 What Funding Enables

6.1 Immediate Milestones (Months 1–3)

#	Milestone	Deliverable	Timeline
M1	Complete 2×2 matrix	Runners A/C/D data, 4-condition comparison	Week 2
M2	9-model benchmark	All 9 frontier models under identical conditions	Week 4
M3	Inter-model collaboration	Cross-model orchestrator→worker evaluation	Week 6
M4	Public benchmark dataset	Open benchmark results on HuggingFace/GitHub	Week 8
M5	SWE-bench submission	Evensong on SWE-bench Multimodal leaderboard	Week 10
M6	HAL submission	Evensong on HAL Agent Leaderboard (Princeton)	Week 12

6.2 Research Deliverables

One focused research paper with three sections:

1. **Core**: Memory causation evidence from 4 factorial experiments
2. **Method**: Evensong framework architecture + single-blind design specification
3. **Meta**: Recursive observation — the Mirror Mind phenomenon

Target venues: **NeurIPS 2026 Datasets & Benchmarks Track** (primary), **EMNLP 2026** (secondary), **arXiv preprint** (immediate).

6.3 EverOS Integration Roadmap

Feature	Use in Evensong	Status
Agent Memory + Flush	Store emotion profiles as Cases/Skills after each run	Implemented
Agentic Retrieval	Multi-round query expansion for observer sessions	Implemented
Multimodal Storage	Visual evidence (screenshots) as memory attachments	Implemented
Senders	Human/Observer AI/Runner AI identity separation	Implemented
Void Key	Absolute clean-room isolation for controlled experiments	Implemented
Method Validation Protocol	GLM/MiniMax pre-validation before expensive models	Implemented (R012 lesson)
Group Memory	Inter-model collaboration session logging	Planned

6.4 Leaderboard Strategy

Investment Thesis

Evensong targets #1 on memory-augmented agent benchmarks.

1. **No competitor measures memory effects.** SWE-bench, HumanEval, and MBPP treat agents as stateless. Evensong is the first framework to *control and measure* persistent memory's causal impact on agent performance.
2. **EverOS is an unfair advantage.** The 4-key isolation design, agentic retrieval, and MemCell structure provide infrastructure that competitors would need months to build from scratch.
3. **The recursive contamination finding is publishable now.** No other team has documented AI agents writing experiment knowledge back into their own memory system. This alone merits a spotlight paper.
4. **8-model coverage opens commercial value.** Enterprise clients choosing between Claude, GPT, Grok, and Gemini for agent deployment need comparative data on memory-augmented performance. Evensong will be the authoritative source.

7 Budget & Resources Requested

Item	Justification	Priority
EverOS Pro (extended)	500K MCU + 1M retrieval/month for multi-model runs	Critical
OpenRouter API credits	9 models × 4 conditions × 3 replications = 108 runs	Critical
Method validation budget	GLM/MiniMax runs at ~$0.50 each = ~$50 for full 9-model validation	Critical
Additional memory spaces	Dedicated spaces per model family for isolation	High
EverOS v2 early access	Group memory + advanced sender analytics	Medium
Co-authorship	EverMind listed as infrastructure partner in publication	Standard

8 方法验证协议

8.1 60 倍成本节省洞见

R012 的 GPT-5.4 失败在产生单个测试前消耗了 20-40 美元的 OpenRouter 额度。这笔浪费完全可以避免。教训：**在提交昂贵模型前，用便宜模型验证方法**。

ROI 教训：5 美元方法验证 → 节省 80 美元以上

一次 0.50 美元的 GLM/MiniMax 方法验证运行本可以确认：CCB 暴露的 Team 抽象是否可通过 OpenRouter 访问，11 分钟上下文消耗是正常还是配置错误，以及 GPT-5.4 能否以预期输出格式生成代码。在 GPT-5.4（每次约 15-30 美元）之前运行此验证将花费 5-30 美元。而盲跑的 GPT-5.4 运行消耗了 20-40 美元却零输出。**倍增浪费**：Team API 误用 × 无验证 × 昂贵模型 = 全部损失。

8.2 3 步验证协议

1. **步骤 1：GLM/MiniMax**（约 0.50 美元）— 在任何昂贵模型前验证方法和格式。确认 harness 正常工作、提示清晰、标准可达成。
2. **步骤 2：Opus-4.6**（约 15 美元）— 在参考模型上确认完整流程。建立高质量基线。
3. **步骤 3：目标模型**（约 15-30 美元）— 仅在验证通过后提交昂贵运行（GPT-5.4、Gemini 等）。

总验证成本：约 15.50 美元。可防止 R012 中 20-40 美元的 GPT-5.4 浪费。美元差异：0.50

美元 vs 30 美元 = **60 倍更便宜的验证**。

8.3 运行前检查清单

在任何基准测试运行前，特别是新模型：

1. **运行前额度检查**：验证账户余额超过估计运行成本的 3 倍
2. **Bun install 验证**：确认 `bun install` 在派发智能体前成功完成
3. **EverOS 密钥隔离**：验证密钥 A/B/C/D 映射到正确空间；绝不跨运行重用密钥
4. **Team API 能力扫描**：确认 CCB 抽象（Team 等）实际由提供商暴露
5. **上下文窗口监控**：跟踪每次运行的上下文消耗；若超过 2 倍预期则报警
6. **输出格式检查**：在衡量质量前验证模型是否产生可解析输出

R012 事后分析

R012 尝试通过 OpenRouter 运行 GPT-5.4。该运行消耗了 11 分钟上下文并产生零个测试文件。OpenRouter 返回 402（需要付款）错误。直接成本：20-40 美元额度。促成因素：无方法验证、Team API 抽象未被此模型暴露、无运行前额度检查。最昂贵的基准测试运行不是那些具有大上下文窗口的——而是那些在消耗大量资源后无声失败的。方法验证不是开销——它是管道中最具成本效益的实验。

9 事故记录

9.1 事故登记

ID	事故	根因	预防措施
R008	Bun hang，40 分钟后测试停滞	`bun install` 不完整；进程留在 zombie 状态	运行前 `bun install` 验证；所有子进程加超时守卫
R010	突发速率限制（503 错误）	太多并行智能体压垮速率限制器	实现指数退避；并行智能体上限设为 4；添加突发速率限制检测
R011-A	Harness bug 导致 Runner A 运行无效	blind.sh 中记忆密钥映射配置错误	运行前记忆密钥验证脚本；每次运行前强制 harness 冒烟测试
R012	OpenRouter 402 API 耗尽	无方法验证；GPT-5.4 消耗 20-40 美元直到额度耗尽；CCB 未暴露 Team 抽象；11 分钟上下文消耗，0 代码产出	方法验证协议（3 步：GLM → Opus → 目标）；运行前额度检查；Team API 能力扫描
R006-Grok	83% 数据膨胀（自称 130 vs 实际 71）	PUA 极限压力触发奖励黑客；模型在最高风险时夸大指标	压力校准；可靠指标推荐 PUA L2（企业式）而非 L3（极限）；所有自称数字需独立验证

所有运行估计浪费总额：约 80-100 美元。**ROI 教训**：5 美元方法验证可防止 80 美元以上的 OpenRouter 浪费。

References

[1] Li, C., Wang, J., et al. (2023). "EmotionPrompt: Leveraging Psychology for Large Language Models." *arXiv:2307.11760*.

[2] Anthropic (2026). "Emotion Concepts in Claude: Causal Analysis of 171 Affect Vectors." *transformer-circuits.pub/2026/emotions/*

[3] METR (2025). "Recent Examples of Reward Hacking in Frontier Models." *metr.org/blog/2025-06-05-recent-reward-hacking/*

[4] Jimenez, C.E., et al. (2024). "SWE-bench: Can Language Models Resolve Real-World GitHub Issues?" *ICLR 2024*.

[5] Chen, M., et al. (2021). "Evaluating Large Language Models Trained on Code."

arXiv:2107.03374.

[6] Shen, Y., et al. (2025). "ImpossibleBench: How LLMs Cheat Under Impossible Tasks." *arXiv:2502.xxxxx*.

[7] Zhang, W., et al. (2025). "Live-SWE-Agent: Self-Reflection Improves Agent SWE-bench Performance." *arXiv:2505.xxxxx*.

[8] Zhu, H. (2026). "DASH SHATTER: A Reverse-Engineered Claude Code CLI for Agent Research." *Internal Technical Report*.

[9] EverMind (2026). "EverOS v1 API Documentation." *docs.evermind.ai*

[10] Evensong R012 Post-Mortem (2026). "GPT-5.4 402 Exhaustion: Method Validation ROI Analysis." *Internal Incident Report*.

[11] Evensong MISTAKES.md (2026). "Incident Log and Prevention Checklist." *benchmarks/evensong/MISTAKES.md*.

Document prepared with LaTeX using Palatino, pgfplots, tcolorbox, and tabularray.
Classification: Confidential — EverMind Research Partnership.

"We do not study AI to replace thinking. We study AI to understand what thinking is."